

Lab - Check the Application is Connecting to MySQL

To Begin with the Lab

Summary

This lab demonstrates how to connect a **.NET web application** to a **MySQL database running inside an Azure Container Instance (ACI)**. The application uses the **MySQL Connector for .NET** to establish a connection, retrieve records from the **CourseOrders** table, and display them on a web page, confirming that the containerized MySQL database is functioning correctly.

Understanding

After deploying a custom MySQL container, the next step is to verify that applications can connect to it. The .NET application uses the **MySQL Connector** package to establish a connection to the MySQL server using its **public IP address, port number (3306), database name, username, and password**.

The application executes SQL queries against the MySQL database and displays the retrieved data on an Orders page. If the records appear successfully, it confirms that:

- The MySQL container is running.
- The database is initialized correctly.
- Network connectivity is working.
- The application can communicate with the database.

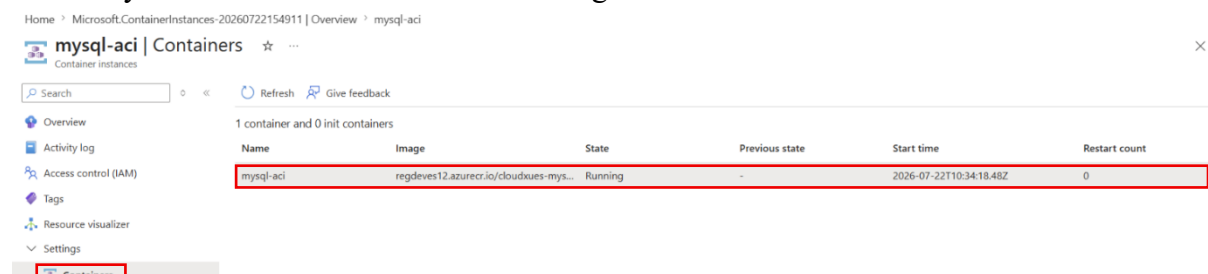
Example:

- Browser → ASP.NET Application
- ASP.NET Application → MySQL Connector
- MySQL Connector → Azure Container Instance (MySQL)
- MySQL Database → Returns CourseOrders data

Lab Steps

Step 1: Verify the MySQL Container

- Open the **Azure Portal**.
- Navigate to the **Azure Container Instance**.
- Open **Containers**.
- Verify that the container status is: **Running**

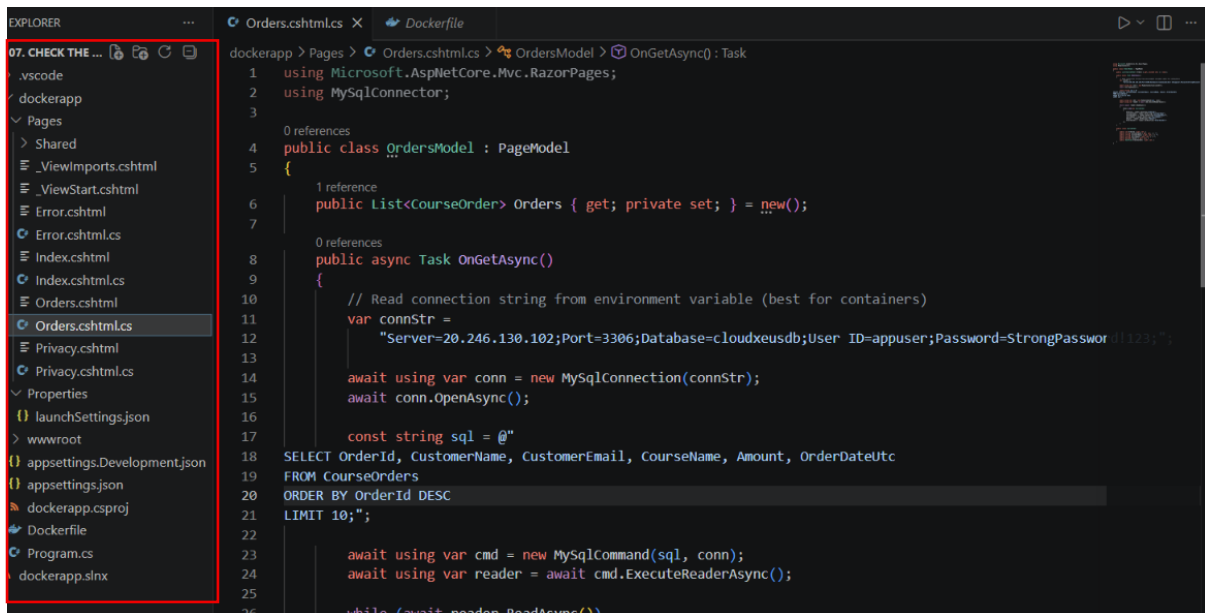


The screenshot shows the Azure Portal interface for a MySQL container instance. The breadcrumb navigation at the top reads: Home > Microsoft.ContainerInstances-20260722154911 | Overview > mysql-aci. The main heading is 'mysql-aci | Containers'. Below this, a table lists the container instances. The table has columns: Name, Image, State, Previous state, Start time, and Restart count. A single row is shown with the name 'mysql-aci', image 'regdevs12.azurecr.io/cloudxues-mys...', state 'Running', previous state '-', start time '2026-07-22T10:34:18.48Z', and restart count '0'. The 'Containers' tab in the left-hand navigation menu is highlighted.

Name	Image	State	Previous state	Start time	Restart count
mysql-aci	regdevs12.azurecr.io/cloudxues-mys...	Running	-	2026-07-22T10:34:18.48Z	0

Step 2: Open the .NET Project

- Open the existing **DockerApp** (.NET Web Application).
- Verify that the project contains an **Orders** page.

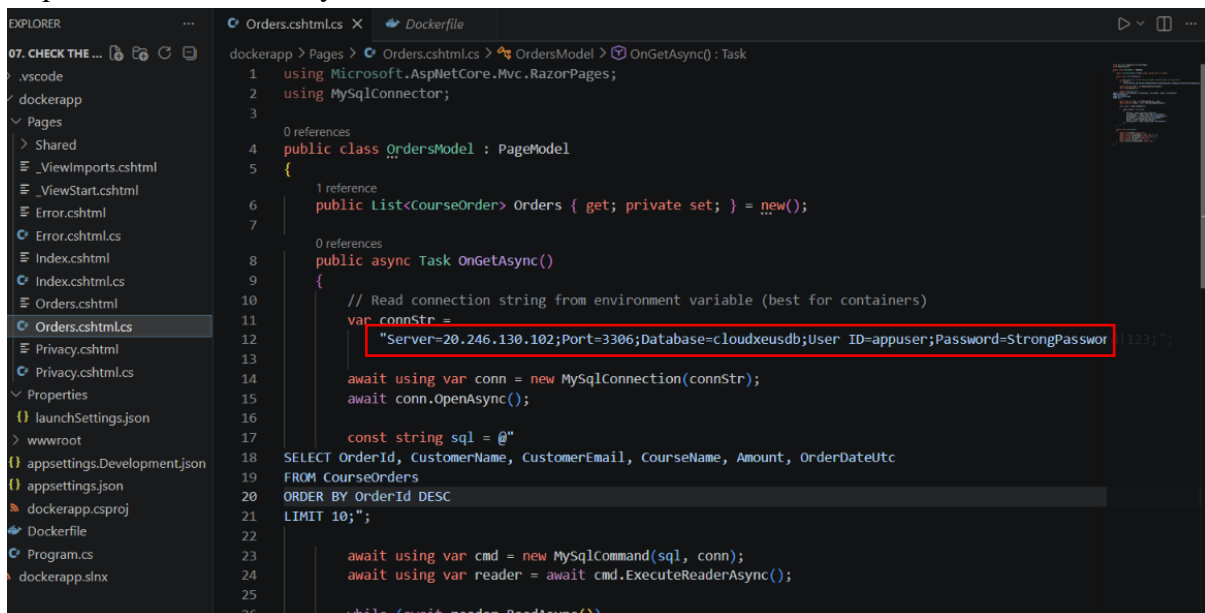


Step 4: Configure the Connection String

Configure the connection string with:

- Server = Public IP of the MySQL Container
- Port = 3306
- Database Name
- Username
- Password

Replace the values with your own container details.



Step 8: Build the Application

- Open the Terminal.
- Run:

dotnet build

Verify that the build completes successfully.

```
PS D:\SIMPLE APP\code\dockerapp> dotnet build
Restore complete (9.5s)
  dockerapp net10.0 succeeded (5.0s) -> bin\Debug\net10.0\dockerapp.dll

Build succeeded in 15.3s
PS D:\SIMPLE APP\code\dockerapp>
```

Step 9: Run the Application

Run:

```
dotnet run
```

- Copy the **localhost**

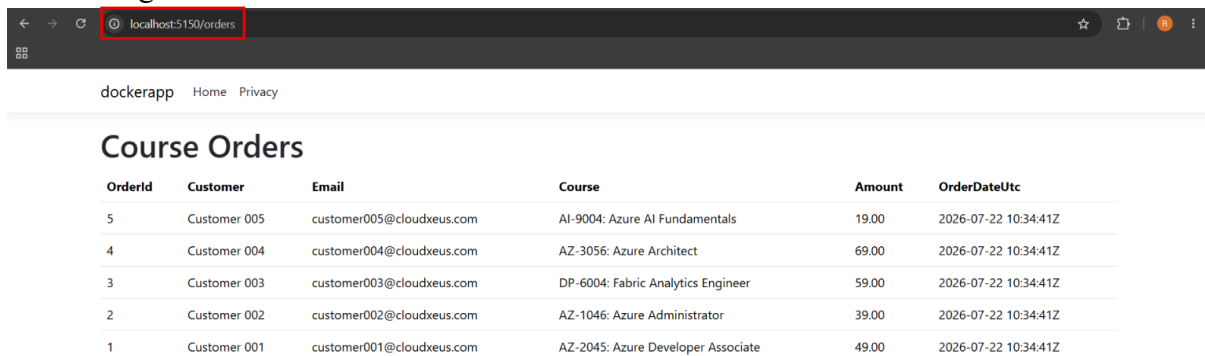
```
PS D:\SIMPLE APP\code\dockerapp> dotnet run
Using launch settings from D:\SIMPLE APP\code\dockerapp\Properties\launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://localhost:5150
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: D:\SIMPLE APP\code\dockerapp
```

Step 10: Open the Application

- Open the application in your browser.
- Paste the **localhost**.

Step 12: Verify the Output

- Confirm that the **CourseOrders** records are displayed.
- This verifies that the .NET application is successfully connected to the MySQL database running inside the Azure Container Instance.



Orderid	Customer	Email	Course	Amount	OrderDateUtc
5	Customer 005	customer005@cloudxus.com	AI-9004: Azure AI Fundamentals	19.00	2026-07-22 10:34:41Z
4	Customer 004	customer004@cloudxus.com	AZ-3056: Azure Architect	69.00	2026-07-22 10:34:41Z
3	Customer 003	customer003@cloudxus.com	DP-6004: Fabric Analytics Engineer	59.00	2026-07-22 10:34:41Z
2	Customer 002	customer002@cloudxus.com	AZ-1046: Azure Administrator	39.00	2026-07-22 10:34:41Z
1	Customer 001	customer001@cloudxus.com	AZ-2045: Azure Developer Associate	49.00	2026-07-22 10:34:41Z